SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Mini LeetCode Coding Platform

Course Project for Software Technology

Document Version: 1.0.0 | June 8, 2026

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

FOR MINI LEETCODE CODING PLATFORM

Course Project for Software Technology

Document Version: 1.0.0

Release Date: June 8, 2026

Status: Approved Specification

TABLE OF CONTENTS

---

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) document details the functional, nonfunctional, design, and interface requirements for the **Mini LeetCode Coding Platform** v1.0.0. The platform is designed as an educational tool for computer science students to practice coding challenges in JavaScript and Python 3. This specification acts as a binding contract and reference guide for evaluation and development.

1.2 Document Conventions

This document follows the standard **IEEE Std 830-1998** structure for software requirements specifications. Formatting highlights:

- **Bold text** represents core architectural components or terminology.
- `Monospace text` indicates code symbols, functions, variables, file names, or URLs.
- Highlighted blocks represent important design constraints or specifications.

1.3 Intended Audience and Reading Suggestions

The primary audience includes:

- **Project Evaluators & Professors** (Course Software Technology project verification).
- **Developers** (For expanding compilers or visual tools).
- **Users/Students** (For deployment instructions and platform features).

The reader should start with Section 2 to understand the product scope, progress to Section 4 for detailed functional requirements, and consult Section 6 for engineering details.

1.4 Product Scope

Mini LeetCode is a client-side Single Page Application (SPA). The product provides:

- A searchable table of algorithm challenges.
- A rich text workspace containing problem descriptions, constraints, examples, and Monaco Editor.
- An in-browser safe sandboxed compiler for JavaScript and Python.
- A custom problem creator storing data locally to allow expansion.
- Local performance logs and analytics charts.

1.5 References

1. *IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.*

2. *Pyodide WebAssembly Compilation Specifications (https://pyodide.org).*

3. *Monaco Editor AMD Loader Architecture (https://microsoft.github.io/monaco-editor).*

---

2. OVERALL DESCRIPTION

2.1 Product Perspective

Mini LeetCode operates as an offline-first, client-side application running directly inside the user's web browser. Unlike traditional online judges (e.g., LeetCode, HackerRank) that require expensive remote servers, compilers, and Docker containers to run user submissions, Mini LeetCode performs all processes client-side. This layout is cost-efficient, highly responsive, and portable.

2.2 Product Functions

The high-level functions of the system include:

- **Dashboard Management**: Lists algorithm challenges, displays user progress charts, and tracks active day streaks.
- **Algorithm Filtering**: Dynamically filters questions by difficulty (Easy, Medium, Hard), Category (Arrays, Strings, Math), and completion status.
- **Code Authoring**: Provides a full IDE-like authoring workspace with bracket closing, autocomplete, and theme switches.
- **Sandboxed Compilation**: Safely executes user-defined JavaScript and Python scripts in independent background threads, preventing browser tab freezes.
- **Execution Reporting**: Intercepts `console.log()` and `print()` output, measures execution time, and checks results against test assertions.
- **Challenge Extension**: Offers forms to write new coding challenges with custom parameter schemas, which are integrated into the main database.

2.3 User Classes and Characteristics

The platform caters to two primary user profiles:

1. **Student / Programmer (Primary User)**: Seeks a responsive, zero-latency coding platform to practice common algorithms. Requires syntax highlighting, autocomplete, and stdout output views.

2. **Educator / Administrator**: Needs to add custom coding problems or test sets. Requires a form-based interface that parses inputs into JSON structures.

2.4 Operating Environment

The application operates inside standard modern web browsers. Minimum environments:

- **Microsoft Edge** (v90+)

- **Google Chrome** (v90+)
- **Mozilla Firefox** (v88+)
- **Safari** (v14+)
- **Operating Systems**: Windows 10/11, macOS, Linux, ChromeOS.
- **CORS Bypass**: Operates successfully over local filesystem protocols (`file:///`) using inline worker Blobs.

2.5 Design and Implementation Constraints

- **Zero-Compile Architecture**: The app must use standard Vanilla HTML5, CSS3, and ES6 JS without compiling/bundling frameworks (such as React, Vue, or Webpack) to ensure it runs by double-clicking `index.html`.
- **Browser-Only Execution**: No external API calls are allowed for compiling code. All execution must occur locally inside Web Workers.
- **CDN Dependency**: External assets (Monaco, Lucide, Pyodide, Confetti) are loaded via reliable CDNs. An active internet connection is required on the first load, after which the browser caches them.
- **Storage Limits**: Local browser storage (`localStorage`) is capped at 5MB, which is sufficient for thousands of user submissions and custom problems.

2.6 Assumptions and Dependencies

- **Browser Capability**: Assumes the user's browser supports HTML5 Web Workers and WebAssembly (WASM).
- **CDN Availability**: Assumes that standard CDNs (cdnjs, unpkg, jsdelivr) are accessible.
- **Parameters Validation**: Assumes that users provide valid JSON representations in testcases and custom problem parameters.

---

3. EXTERNAL INTERFACE REQUIREMENTS

3.1 User Interfaces

The user interface follows a modern dark-themed glassmorphism layout, prioritizing visual hierarchy and micro-interactions. Key components:

- **Global Typography**: Standardized on **Outfit** for interface texts and **JetBrains Mono** for code scripts.
- **Responsive Navigation**: Navbar contains active-status buttons, user avatar, and an animated flame icon for the daily streak badge.
- **Split-Pane Workspace**: A 2-column layout. The left column (460px fixed width) hosts description details and submissions history. The right column takes the remaining space, split horizontally between Monaco Editor and the console drawer.
- **Console Slider**: Sits at the bottom of the workspace. Expands to 320px on chevron/header click and collapses to 98px to maximize editor room.

3.2 Hardware Interfaces

The application does not interact directly with hardware interfaces. It operates through standard browser APIs mapping to local system resources (CPU cores for Web Workers, RAM for Pyodide WebAssembly).

3.3 Software Interfaces

The application interfaces with the following libraries:

Library Name	Version	Interface Method	Purpose

3.4 Communications Interfaces

- **HTTP / HTTPS**: Used exclusively to pull CDN dependencies on initial startup.
- **Local Protocol (`file:///`)**: Supported for server-less operations. The application uses inline Web Worker Blobs to bypass security blocks on the local filesystem.

---

4. SYSTEM FEATURES

4.1 Problems Dashboard & Real-Time Search

- **Description**: The central directory displaying all default and user-created coding questions.
- **Functional Requirements**:
- Search input updates table lists instantly based on text matches in title or category.
- Dropdown selectors filter problems by Difficulty, Category, and status.
- Selecting **"Solve"** loads the problem into the Workspace view and changes routes.

4.2 Interactive Code Editor Pane

- **Description**: A wrapper holding the Monaco Editor for code writing.
- **Functional Requirements**:
- Dynamically updates language models between JavaScript and Python on language selector change.
- Displays Pyodide engine initialization status while loading the WebAssembly interpreter.
- Saves user drafts in localStorage on every edit, preventing progress loss on page reloads.
- Reset button resets code back to default starter templates.

4.3 Collapsible Execution Console Drawer

- **Description**: Displays parameters inputs and results outputs.
- **Functional Requirements**:
- Displays input text fields matching the function's parameters.
- Triggers sequential worker execution on **"Run Code"** or **"Submit"** click.
- Shows runtime loaders and outputs.
- Displays stdout print messages and expected vs actual diff comparisons.
- Highlights error stack tracelines on runtime or compile issues.

4.4 Custom Problem Creator Wizard

- **Description**: A form-based creator permitting users to build custom questions.
- **Functional Requirements**:
- Fields validation for title, difficulty, category, parameter lists, function name, and starter codes.
- Parses comma-separated arguments and JSON representations of test case inputs/expecteds.
- Save action appends the problem to the list, updates statistics, and routes back to the dashboard.

4.5 Statistics Dashboard & Analytics

- **Description**: A graphical panel reporting user metrics.
- **Functional Requirements**:
- Aggregates solved counts by difficulty and calculates completion percentages.
- Animates SVG circular progress meters.
- Charts submission success-to-failure ratios.
- Renders recent submission history logs with code restoration options.

4.6 Daily Streak & Local Storage Persistence

- **Description**: Tracks user activity and persists system states.
- **Functional Requirements**:
- Increments streak count if a user accesses the site on a new calendar day. Resets if a day is skipped.
- Saves drafts, submissions history, and custom problems in JSON structures in `localStorage`.

---

5. OTHER NONFUNCTIONAL REQUIREMENTS

5.1 Performance Requirements

- **Page Load Speed**: The initial HTML structure and design style must load in under 1 second.
- **Monaco Load**: Monaco Editor initialization must load within 2 seconds.
- **JavaScript Execution**: JavaScript compilation and runtimes must execute within 150ms for normal cases.
- **Python Execution**: Pyodide loader should initialize in under 5 seconds on first use, after which script executions must compile in under 100ms.
- **UI Responsiveness**: Filters and search queries must update the problems list table in under 50ms.

5.2 Safety Requirements

- **Infinite Loop Protection**: All running codes must terminate using strict watchdog timers. JavaScript workers are terminated after 3000ms. Python workers are terminated after 8000ms.
- **Browser Stability**: Code runner calls must run in independent threads to ensure that infinite loops do not block browser tabs or cause app crashes.

5.3 Security Requirements

- **No Remote Shell Access**: Because execution runs entirely in the user's browser, there is no risk of remote shell injection attacks on a server.
- **Sandboxed Scope**: Workers run without access to the document object model (DOM), preventing malicious user code from executing cross-site scripting (XSS) modifications on the interface elements.

5.4 Software Quality Attributes

- **Portability**: Code must be fully self-contained and run on any browser without installer packages or environment configs.
- **Usability**: Responsive layouts must accommodate standard laptops, desktops, and tablet screens.
- **Reliability**: Code runner must recover from Pyodide exceptions or worker failures by rebuilding the worker context automatically on subsequent executions.

---

6. DETAILED ARCHITECTURE & CODE LAYOUT

6.1 Script Loading Protocol & CORS Bypass

To bypass local file CORS policy blocks, the application converts worker script strings into Blobs:

```javascript
const blob = new Blob([jsWorkerCode], { type: 'application/javascript' });
const worker = new Worker(URL.createObjectURL(blob));
```

This encapsulates execution scripts inside local origin objects, allowing workers to run smoothly over the `file:///` protocol.

6.2 Web Worker Isolation Engine

The execution model isolates compiled codes using the following sequence:

```
[Main Thread: btn-run-code Click]

v

[Read Monaco Code & Parameters]

v

[Instantiate Web Worker Blob]

+----------------------+

v v

[JS Worker Thread] [Pyodide Worker Thread]

v v
```

```
[Evaluate code] [Import Pyodide WASM]

| v

v [Run code in WebAssembly]


+-----------+-----------+


v

[Main Thread Watchdog: Timeout reached?]


+------------+------------+

| Yes | No

v v

[Terminate Worker] [Return Output, logs, runtime]


v v

[Display TLE Error] [Update Result panel]
```

6.3 State Schema Definition

The system state is structured within the following JSON layouts:

Submission Entry Schema:

`json

{

"id": "sub_1717873618192",

"problemId": "two-sum",

"problemTitle": "Two Sum",

"language": "javascript",

"status": "Accepted",

"runtime": "12.45 ms",

"code": "function twoSum(nums, target) { ... }",

"timestamp": "2026-06-08 14:15:20"

}

`

Custom Problem Schema:

`json

{

"id": "fizz-buzz",

"title": "Fizz Buzz",

"difficulty": "Easy",

"category": "Math",

"description": "Write a program that outputs...",

"constraints": ["1 <= n <= 10^4"],

"examples": [

```
{ "input": "n = 3", "output": "[\"1\",\"2\",\"Fizz\"]" }

],

"starterCode": {

"javascript": "function fizzBuzz(n) {\n\n}",

"python": "def fizzBuzz(n):\n pass"

},

"functionName": "fizzBuzz",

"paramNames": ["n"],

"testCases": [

{ "input": [3], "expected": ["1","2","Fizz"] }

],

"validationCases": []

}
```

---

7. TRACEABILITY MATRIX & VERIFICATION SUITE

7.1 Requirement to Feature Mapping

Req ID	Requirement Description	System Feature	Implementation Method

7.2 Validation Case Scenarios

1. **Assertion Verification**: Running the Two Sum solver returns matching outputs. The console logs "Case 1: Passed" and displays expected and actual return values.

2. **Infinite Loop Termination**: Submitting `while(true) {}` in JavaScript terminates after exactly 3000ms. The output console displays a red error header: **"Time Limit Exceeded (TLE)"** and avoids tab freezing.

3. **Draft Synchronization**: Editing the editor window, closing the tab, and reloading the page restores the draft code, verifying state serialization in localStorage.